https://github.com/AnuragHarapanahalli/cnl-sem5/blob/main/3.PY

https://github.com/AnuragHarapanahalli/cnl-sem5/blob/main/3-new.py

```
# Program to identify IP Class, Range, Default Subnet Mask,
# Number of Networks, Number of Hosts, and Binary Reasoning

ip = input("Enter an IP address: ")

first_octet = int(ip.split(".")[0])

if 1 <= first_octet <= 126:
    print("\nClass A")
    print("Range: 1.0.0.0 - 126.255.255.255")
    print("Binary Reason:")
    print("  First bit is fixed as 0")
    print("  Pattern: 0xxxxxxx")
    print("Default Subnet Mask: 255.0.0.0 (/8)")
    print("No of Networks: 2^7 - 2 = 126")
    print("Reason:")
    print("  - First bit is fixed as 0, leaving 7 bits for network IDs.")
    print("  - Therefore, Number of Networks = 2^7.")
    print("  - Subtract 2 because Network 0 is reserved and Network 127 is reserved for Loopback.")
    print("No of Hosts: 2^24 - 2 = 16,777,214")
    print("Reason:")
    print("  - 24 bits are available for host IDs.")
```

```
    print("  - Subtract 2 because one address is the Network Address and one is the Broadcast Address.")

elif first_octet == 127:
    print("\nLoopback Address")
    print("Range: 127.0.0.0 - 127.255.255.255")
    print("Binary: 01111111")
    print("Reason:")
    print("  - Reserved by the IPv4 standard for loopback testing.")
    print("  - Used to test the local machine.")
    print("  - Any packet sent to a 127.x.x.x address is returned to the same computer instead of being sent over the network.")

elif 128 <= first_octet <= 191:
    print("\nClass B")
    print("Range: 128.0.0.0 - 191.255.255.255")
    print("Binary Reason:")
    print("  First two bits are fixed as 10")
    print("  Pattern: 10xxxxxx")
    print("Default Subnet Mask: 255.255.0.0 (/16)")
    print("No of Networks: 2^14 = 16,384")
    print("Reason:")
    print("  - First 2 bits are fixed as 10.")
    print("  - Remaining 6 bits in the first octet + 8 bits in the second octet = 14 network bits.")
    print("  - Therefore, Number of Networks = 2^14.")
    print("No of Hosts: 2^16 - 2 = 65,534")
    print("Reason:")
    print("  - 16 bits are available for host IDs.")
```

```
    print("  - Subtract 2 because one address is the Network Address and one is the Broadcast Address.")

elif 192 <= first_octet <= 223:
    print("\nClass C")
    print("Range: 192.0.0.0 - 223.255.255.255")
    print("Binary Reason:")
    print("  First three bits are fixed as 110")
    print("  Pattern: 110xxxxx")
    print("Default Subnet Mask: 255.255.255.0 (/24)")
    print("No of Networks: 2^21 = 2,097,152")
    print("Reason:")
    print("  - First 3 bits are fixed as 110.")
    print("  - Remaining 5 bits in the first octet + 8 bits in the second octet + 8 bits in the third octet = 21 network bits.")
    print("  - Therefore, Number of Networks = 2^21.")
    print("No of Hosts: 2^8 - 2 = 254")
    print("Reason:")
    print("  - 8 bits are available for host IDs.")
    print("  - Subtract 2 because one address is the Network Address and one is the Broadcast Address.")

elif 224 <= first_octet <= 239:
    print("\nClass D (Multicast)")
    print("Range: 224.0.0.0 - 239.255.255.255")
    print("Binary Reason:")
    print("  First four bits are fixed as 1110")
    print("  Pattern: 1110xxxx")
    print("Default Subnet Mask: Not Applicable")
```

```
    print("No of Networks: Not Applicable")
    print("No of Hosts: Not Applicable")
    print("Reason:")
    print("  - Class D addresses are reserved for multicast communication.")
    print("  - They are not assigned to individual hosts.")

elif 240 <= first_octet <= 255:
    print("\nClass E (Experimental)")
    print("Range: 240.0.0.0 - 255.255.255.255")
    print("Binary Reason:")
    print("  First four bits are fixed as 1111")
    print("  Pattern: 1111xxxx")
    print("Default Subnet Mask: Not Applicable")
    print("No of Networks: Not Applicable")
    print("No of Hosts: Not Applicable")
    print("Reason:")
    print("  - Class E addresses are reserved for experimental and research purposes.")
    print("  - They are not used for general host addressing.")

else:
    print("Invalid IP Address")
```

CODE 2

code:

```
#include <iostream>
#include <cmath>
```

```
#include <string>

using namespace std;

int main() {
   int o1, o2, o3, o4;
   char dot;
   int cidr;

   // 1. Get user input for Base IP and CIDR prefix
   cout << "Enter base IP address (e.g., 192.168.10.0): ";
   cin >> o1 >> dot >> o2 >> dot >> o3 >> dot >> o4;

   cout << "Enter CIDR prefix (e.g., 26 for /26): ";
   cin >> cidr;

   // Validate for Class C subnetting range
   if (cidr < 24 || cidr > 30) {
      cout << "Error: This program currently supports Class C subnetting (CIDR 24 to 30)."
<< endl;
      return 1;
   }

   // 2. Calculate the core subnetting details
   int hostBits = 32 - cidr;
   int blockSize = pow(2, hostBits);
   int numSubnets = 256 / blockSize;
   int usableHosts = blockSize - 2;
```

```
    int lastOctetMask = 256 - blockSize;

    // Display general subnet information
    cout << "\n--- General Subnet Information ---" << endl;
    cout << "Calculated Subnet Mask : 255.255.255." << lastOctetMask << endl;
    cout << "Total Usable Hosts     : " << usableHosts << " per subnet" << endl;
    cout << "Total Subnets Created  : " << numSubnets << "\n" << endl;

    // 3 & 4. Calculate and Display specifics for each subnet
    for (int i = 0; i < numSubnets; ++i) {
        int network = i * blockSize;
        int firstHost = network + 1;
        int lastHost = network + usableHosts;
        int broadcast = network + blockSize - 1;

        cout << "=========================================" << endl;
        cout << "Subnet " << (i + 1) << endl;
        cout << "=========================================" << endl;
        cout << "Network Address   : " << o1 << "." << o2 << "." << o3 << "." << network << endl;
        cout << "First Host        : " << o1 << "." << o2 << "." << o3 << "." << firstHost << endl;
        cout << "Last Host         : " << o1 << "." << o2 << "." << o3 << "." << lastHost << endl;
        cout << "Broadcast Address : " << o1 << "." << o2 << "." << o3 << "." << broadcast <<
endl;

        // Print all usable IP addresses in this subnet
        cout << "\nUsable IP Addresses:" << endl;
        for (int j = firstHost; j <= lastHost; ++j) {
            cout << "  " << o1 << "." << o2 << "." << o3 << "." << j << endl;
```

```
    }
    cout << endl;
  }

  return 0;
}
```



